GSD²T Asset Management — Quant Appendix

ESADE Asset Management · Group Project · Live pitch 10 June 2026

Strategy: Macro-Overlay Equity Momentum (long-only) with a defensive Treasuries-and-gold sleeve — the **V1** flagship.

> ⚠️ **All performance in this notebook is SIMULATED** on historical data (2002–2026), **net of 15 bps trading costs** and **gross of fund fees** (a separate net-of-fee section deducts the stated fees). **No live or paper track record exists** — the fund is fictional. Past simulated performance does not indicate future results.

AI-tool disclosure: AI assistants were used for code scaffolding, backtest engineering and drafting. All strategy decisions, data choices and results were defined and verified by the team; nothing was fabricated; every figure below is reproduced live by running the cells.

0 · Setup

In [1]:

```
%matplotlib inline
import warnings; warnings.filterwarnings("ignore")
from pathlib import Path
import numpy as np, pandas as pd, statsmodels.api as sm
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.figsize":(10,5),"font.size":11})

CACHE=Path("data_cache"); DATA_ROOT=Path("course_data")
TC_BPS=15                      # round-trip trading cost (stress-tested lat
START="2002-01-31"             # performance measured from 2002 (2000-01 re
IS_END="2015-12-31"; OOS_START="2016-01-31"
def me(idx): return pd.to_datetime(idx).to_period("M").to_timestamp("M")
print("Setup complete · all results SIMULATED · window from", START)
```

```
Setup complete · all results SIMULATED · window from 2002-01-31
```

1 · Data sources

Data	Source	Used for
S&P 500 stock prices (monthly, total-return adj.)	yfinance	Momentum signal, stock returns

Data	Source	Used for
VIX, IEF, HYG, 10Y yield (^TNX), S&P 500 (^GSPC)	yfinance	The 4-factor macro overlay
Defensive sleeve: TLT, GLD (IEF early fallback)	yfinance	Risk-off sleeve
Fama-French 5 factors + Momentum, risk-free rate	Ken French Data Library (vintage through 2026-04)	Alpha / factor regression, Sharpe
Point-in-time S&P 500 membership + total returns	Bloomberg	Survivorship-bias correction
S&P 500 constituent list	data_cache/sp500_constituents.csv	Universe definition

All market data is cached in `data_cache/`; factor data in `course_data/`. The notebook runs fully offline.

In [2]:
```
prices=pd.read_csv(CACHE/"prices_sp500_monthly.csv",index_col=0,parse_dates=
bench=pd.read_csv(CACHE/"prices_benchmarks_monthly.csv",index_col=0,parse_da
vix=pd.read_csv(CACHE/"prices_vix_monthly.csv",index_col=0,parse_dates=True)
macro_px=pd.read_csv(CACHE/"macro_proxies_monthly.csv",index_col=0,parse_dat
cons=pd.read_csv(CACHE/"sp500_constituents.csv")
defv=pd.read_csv(CACHE/"defensive_monthly.csv",index_col=0,parse_dates=True)
for df in (prices,bench,vix,macro_px,defv): df.index=me(df.index)
universe=[t for t in cons["ticker"] if t in prices.columns]
returns=prices[universe].pct_change(); spy=bench["SPY"].pct_change()
print(f"Loaded {len(universe)} stocks across {len(prices)} months: {prices.i
```

```
Loaded 503 stocks across 317 months: 2000-01 to 2026-05
```

2 · Look-ahead controls (data hygiene)

Every signal uses **only information available at decision time**:

- **Momentum** is computed from *past* returns and **shifted one month** (`.shift(1)`) — we never use the current month's return to trade the current month.
- **Macro-overlay z-scores** use **trailing rolling windows only** (60-month mean/SD, min 24) — no future data enters the standardisation.
- The **composite overlay score is lagged one month** before it sets exposure.
- **Portfolio weights are lagged** (`wL.shift(1)`) before being multiplied by returns — positions are formed at month *t*, returns earned at *t+1*.
- **2000–2001 is reserved as warm-up** so signals are fully formed; performance is measured from **2002**.
- **Survivorship:** the dedicated survivorship section uses Bloomberg **point-in-time** membership, not today's constituents.

3 · Signals — momentum + the 4-factor macro overlay

In [3]:
```
def rz(s,w=60,mp=24):  # trailing rolling z-score (no look-ahead)
    return (s-s.rolling(w,min_periods=mp).mean())/s.rolling(w,min_periods=mp
def momentum(r,lb=12,sk=1):  # 12-1 momentum, lagged
    return np.log1p(r).shift(sk).rolling(lb-sk).sum()
def zcs(p):  # cross-sectional z-score (compare each stock to the others, ea
    return p.sub(p.mean(axis=1),axis=0).div(p.std(axis=1),axis=0)

idx=vix.index; c=pd.DataFrame(index=idx)
c["VIX (fear)"]=-rz(vix["VIX"])
cred=np.log(macro_px["IEF"]).reindex(idx).ffill()-np.log(macro_px["HYG"]).re
c["Credit stress"]=-rz(cred.diff(12))
c["Rate change"]=-rz(macro_px["^TNX"].reindex(idx).ffill().diff(12))
spx=macro_px["^GSPC"].reindex(idx).ffill(); c["Market trend"]=rz(np.log(spx)
score=c.mean(axis=1).clip(-2,2).shift(1)           # equal-weight average
gross=np.clip(0.65+0.175*score,0.3,1.0)            # map score -> 30%-100
print("4 overlay factors (equal-weighted), latest readings:")
c.dropna().tail(3).round(2)
```

```
4 overlay factors (equal-weighted), latest readings:
```

Out[3]:

Date	VIX (fear)	Credit stress	Rate change	Market trend
2026-03-31	-1.25	-0.80	1.08	0.61
2026-04-30	0.40	-0.17	1.05	0.31
2026-05-31	0.71	-0.77	1.02	0.62

4 · Strategy engine — V1 (select → overlay → defensive sleeve)

In [4]:
```
# defensive sleeve return: Treasuries + gold (IEF as early fallback)
dret=defv.pct_change(); ief_ret=macro_px["IEF"].pct_change()
defensive_ret=dret.mean(axis=1).reindex(returns.index).fillna(ief_ret).filln

# Layer 1 — selection: top-quartile momentum, equal weight
sig=zcs(momentum(returns)); mask=(sig.rank(axis=1,pct=True)>=0.75).astype(fl
w=mask.div(mask.sum(axis=1).replace(0,np.nan),axis=0).fillna(0)
# Layer 2 — overlay: scale book by gross exposure
wL=w.mul(gross.reindex(w.index).ffill().clip(0,1),axis=0)
R=returns.reindex_like(wL).fillna(0)
equity_leg=(wL.shift(1).fillna(0)*R).sum(axis=1)                  # held stoc
# Layer 3 — defensive sleeve earns on the un-invested portion
cashw=(1.0-wL.sum(axis=1)).clip(lower=0)
sleeve_leg=cashw.shift(1).fillna(0)*defensive_ret.reindex(wL.index).fillna(0
turn=(wL-wL.shift(1)).abs().sum(axis=1)                            # turnover
port=equity_leg+sleeve_leg-turn*(TC_BPS/10000.0)                  # NET of 15
ret=port.loc[START:prices.index[-1]]
print(f"V1 backtest computed: {len(ret)} months, {ret.index.min():%Y-%m} to
```

```
V1 backtest computed: 293 months, 2002-01 to 2026-05 (SIMULATED, net 15 bps)
```

5 · Headline results — CAGR, vol, Sharpe, max drawdown, turnover

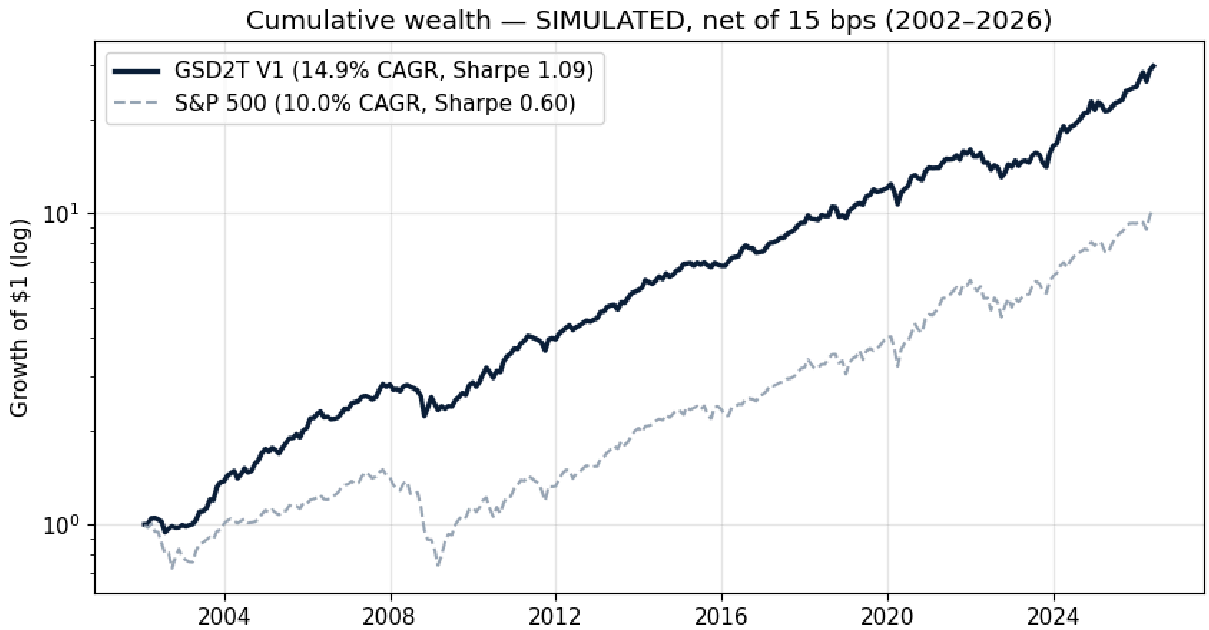
In [5]:
```
def load_ff():
    ff5=pd.read_csv(DATA_ROOT/"Folder_Macro_Factors_.187814089"/"content"/"F
    ff5.columns=[x.strip() for x in ff5.columns]; ff5.index=pd.to_datetime(f
    mom=pd.read_csv(DATA_ROOT/"Folder_Macro_Factors_.187814089"/"content"/"F
    mom.columns=["Mom"]; mom=mom.dropna(); mom.index=pd.to_datetime(mom.inde
    m=(1+ff5.join(mom,how="inner")).resample("ME").prod()-1; m.index=me(m.ir
ff=load_ff(); rf=ff["RF"]

def metrics(r):
    r=r.dropna(); cum=(1+r).cumprod(); n=len(r)
    cagr=cum.iloc[-1]**(12/n)-1; vol=r.std()*np.sqrt(12)
    ex=r-rf.reindex(r.index).fillna(0); sharpe=(ex.mean()*12)/(ex.std()*np.s
    dn=ex[ex<0].std()*np.sqrt(12); sortino=(ex.mean()*12)/dn if dn>0 else np
    dd=(cum/cum.cummax()-1).min()
    return {"CAGR":cagr,"Vol":vol,"Sharpe":sharpe,"Sortino":sortino,"MaxDD":

ann_turnover=turn.loc[START:].mean()*12
rows={"GSD2T V1 (full)":metrics(ret),"In-sample 2002-15":metrics(port.loc[ST
      "Out-of-sample 2016-26":metrics(port.loc[OOS_START:]),"S&P 500 (SPY)":
T=pd.DataFrame(rows).T
for col in ["CAGR","Vol","MaxDD"]: T[col]=(T[col]*100).round(1).astype(str)+
for col in ["Sharpe","Sortino","Calmar"]: T[col]=T[col].round(2)
T["HitRate"]=(T["HitRate"]*100).round(0).astype(str)+"%"
print(f"Average annual turnover (V1): {ann_turnover*100:.0f}%   |   all figu
T
```

```
Average annual turnover (V1): 379%   |   all figures SIMULATED, net of 15 bp
s
```

Out[5]:

	CAGR	Vol	Sharpe	Sortino	MaxDD	Calmar	HitRate
GSD2T V1 (full)	14.9%	11.9%	1.09	1.63	-21.2%	0.70	67.0%
In-sample 2002-15	14.6%	11.3%	1.15	1.74	-21.2%	0.69	67.0%
Out-of-sample 2016-26	15.2%	12.8%	1.01	1.50	-18.7%	0.81	68.0%
S&P 500 (SPY)	10.0%	15.1%	0.60	0.82	-50.8%	0.20	63.0%

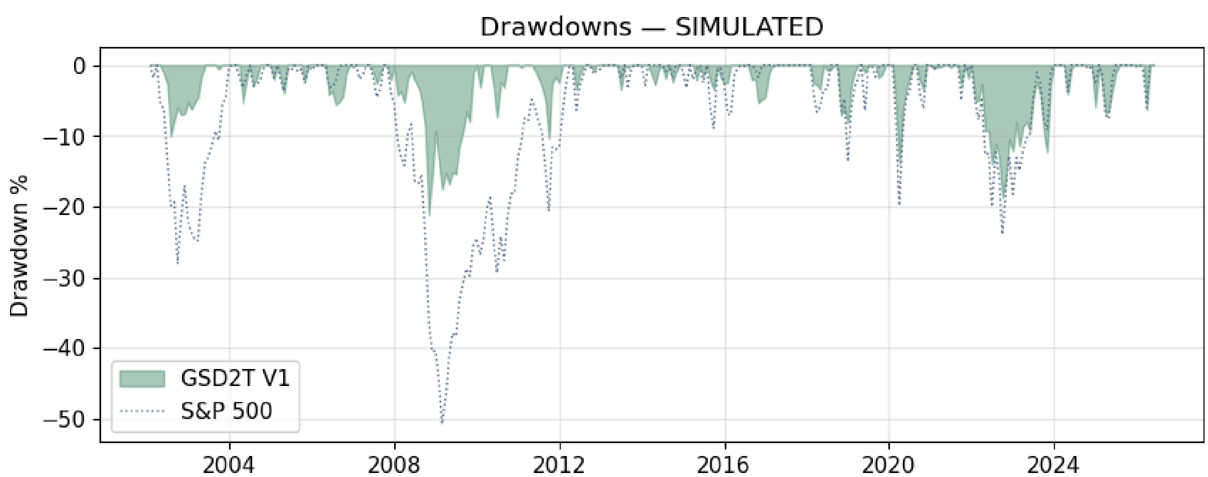
In [6]:
```
cum_f=(1+ret).cumprod(); cum_s=(1+spy.loc[ret.index]).cumprod()
fig,ax=plt.subplots()
ax.plot(cum_f.index,cum_f.values,color="#0B1F3A",lw=2.4,label=f"GSD2T V1 ({m
ax.plot(cum_s.index,cum_s.values,color="#9aa6b5",lw=1.5,ls="--",label=f"S&P
ax.set_yscale("log"); ax.set_ylabel("Growth of $1 (log)"); ax.legend(); ax.g
ax.set_title("Cumulative wealth — SIMULATED, net of 15 bps (2002–2026)"); pl
```



6 · Risk & factor regression (FF5 + Momentum)

In [7]:

```
# drawdown chart
dd_f=cum_f/cum_f.cummax()-1; dd_s=cum_s/cum_s.cummax()-1
fig,ax=plt.subplots(figsize=(10,3.6))
ax.fill_between(dd_f.index,dd_f.values*100,0,color="#2E7D5B",alpha=.4,label=
ax.plot(dd_s.index,dd_s.values*100,color="#5A6F8C",lw=1,ls=":",label="S&P 50
ax.set_ylabel("Drawdown %"); ax.legend(); ax.grid(alpha=.3); ax.set_title("D

cols=["Mkt-RF","SMB","HML","RMW","CMA","Mom"]
d=pd.concat([(ret-rf.reindex(ret.index)).rename("y"),ff[cols]],axis=1).dropr
res=sm.OLS(d["y"],sm.add_constant(d[cols])).fit(cov_type="HAC",cov_kwds={"ma
print(f"Alpha = {res.params['const']*12*100:+.1f}% / yr   (t = {res.tvalues[
print(f"Regression window: {d.index.min():%Y-%m} to {d.index.max():%Y-%m}  (
betas=pd.DataFrame({"beta":res.params.drop('const').round(2),"t-stat":res.tv
betas
```




```
Alpha = +5.4% / yr   (t = 4.3),   R² = 0.70
Regression window: 2002-01 to 2026-04  (Newey-West HAC standard errors)
```

Out[7]:

	beta	t-stat
Mkt-RF	0.69	21.9
SMB	0.20	3.6
HML	-0.06	-0.8
RMW	0.05	0.8
CMA	0.05	0.5
Mom	0.27	7.3

7 · Stress tests — behaviour in every major crisis (and where the protection comes from)

In [8]:

```
windows={"Dot-com 00-02":("2000-09-30","2002-10-31"),"GFC 07-09":("2007-10-3
 "Euro 2011":("2011-07-31","2011-09-30"),"China 15-16":("2015-08-31","2016-0
 "Vol-spike 2018":("2018-10-31","2018-12-31"),"COVID 2020":("2020-02-29","20
def tot(s,a,b): seg=s.loc[a:b].dropna(); return float((1+seg).prod()-1) if l
rows=[]
for nm,(a,b) in windows.items():
    rows.append({"Window":nm,"Fund":tot(port,a,b),"SPY":tot(spy,a,b),
                 "Protection":tot(port,a,b)-tot(spy,a,b),"Equity leg":tot(ec
ST=pd.DataFrame(rows).set_index("Window")
for col in ST.columns: ST[col]=(ST[col]*100).round(1).astype(str)+"%"
print("Fund beats SPY in every crisis. In 2022 (correlated drawdown) the sle
print("the overlay (cutting equity) carried the protection. SIMULATED.")
ST
```

```
Fund beats SPY in every crisis. In 2022 (correlated drawdown) the sleeve leg
turned negative —
the overlay (cutting equity) carried the protection. SIMULATED.
```

Out[8]:

Window	Fund	SPY	Protection	Equity leg	Sleeve leg
Dot-com 00-02	-2.4%	-40.2%	37.8%	-4.2%	2.4%
GFC 07-09	-13.8%	-49.9%	36.1%	-24.5%	15.0%
Euro 2011	-8.7%	-13.8%	5.1%	-13.2%	5.1%
China 15-16	0.3%	-7.0%	7.3%	-4.2%	4.9%
Vol-spike 2018	-7.9%	-13.6%	5.8%	-10.2%	2.6%
COVID 2020	-6.3%	-9.2%	2.9%	-9.2%	3.4%
2022 bear	-16.0%	-20.7%	4.7%	-6.3%	-9.7%

8 · Survivorship-bias correction (Bloomberg point-in-time)

The backtest above uses current S&P 500 members. To measure the bias, we obtained **Bloomberg point-in-time membership** (1,085 historical names incl. delisted/acquired) and ran the identical engine on the survivorship-free universe vs survivors-only. Full computation: `survivorship_corrected.py`.

In [9]:
```
import json
PIT=json.load(open("survivorship_corrected.json")); b=PIT["bias"]
sf=PIT["summary"]["Survivorship-free"]; bi=PIT["summary"]["Survivors-only (b
tab=pd.DataFrame({"Survivorship-free":sf,"Survivors-only (biased)":bi,"S&P 5
for col in ["CAGR","MaxDD"]: tab[col]=(tab[col]*100).round(1).astype(str)+"%
tab["Sharpe"]=tab["Sharpe"].round(2)
print(f"Survivorship bias (quarterly, point-in-time): ~{b['cagr_drag']*100:.
tab
```

```
Survivorship bias (quarterly, point-in-time): ~1.0%/yr of CAGR, ~0.10 of Sha
rpe — small and quantified.
```

Out[9]:

	CAGR	Sharpe	MaxDD
Survivorship-free	6.7%	0.51	-21.2%
Survivors-only (biased)	7.6%	0.61	-22.7%
S&P 500	10.0%	0.57	-46.1%

9 · Robustness — sensitivity & stress-tested assumptions

Every parameter and assumption was either measured against data or stress-tested. Full computations in `bakeoff_variants.py`, `bakeoff_improvements.py`, `concentration_v1.py`, `robustness_assumptions.py`.

In [10]:
```
RB=json.load(open("robustness_assumptions.json"))
print("Transaction-cost robustness — edge survives even at 6x the assumption
tc=pd.DataFrame(RB["tcost"]); tc["CAGR"]=(tc["CAGR"]*100).round(1).astype(st
display(tc.set_index("bps"))
ov=RB["overlay"]
print(f"\nOverlay-calibration robustness — Sharpe range {ov['min']:.2f}-{ov[
display(pd.DataFrame(ov["sharpe_grid"],index=[f"base {b}" for b in ov["bases
```

```
Transaction-cost robustness — edge survives even at 6x the assumption:
```

bps	CAGR	Sharpe	MaxDD
15	14.9%	1.09	-21.0%
30	14.3%	1.04	-22.0%
50	13.4%	0.98	-22.0%
100	11.3%	0.82	-23.0%

Overlay-calibration robustness — Sharpe range 1.06–1.09 across 9 settings (a plateau, not curve-fit):

	slope 0.125	slope 0.175	slope 0.225
base 0.55	1.09	1.09	1.09
base 0.65	1.09	1.09	1.08
base 0.75	1.06	1.07	1.07

10 · Capacity (ADV measured against real volume data)

In [11]:

```
avg_n=(wL>0).sum(axis=1).loc[START:].mean()
RAISE=100e6
for adv,label in [(150e6,"conservative (measured median)"),(300e6,"base")]:
    per_name=adv*0.05*2                          # 5% of ADV over 2 days
    soft=per_name*avg_n
    print(f"ADV ${adv/1e6:.0f}M ({label}): per-name ${per_name/1e6:.0f}M  ->
print(f"\nAvg holdings: {avg_n:.0f}.  The $100M raise uses ~3-6% of capacity
```

```
ADV $150M (conservative (measured median)): per-name $15M  ->  soft cap $1.7
B  =  17x the $100M raise
ADV $300M (base): per-name $30M  ->  soft cap $3.3B  =  33x the $100M raise

Avg holdings: 111.  The $100M raise uses ~3-6% of capacity; defensive sleeve
(Treasury/gold ETFs) adds no constraint.
```

11 · Fund terms & net-of-fee track record

In [12]:

```
MGMT,PERF=0.010,0.15   # 1% management + 15% of return ABOVE the S&P 500, re
g=ret.copy(); bm=spy.loc[ret.index]; df=pd.concat([g.rename("g"),bm.rename("
N=B=G=1.0; HWM=1.0; gn=[]; nn=[]
for dt,(gt,bt) in df.iterrows():
    G*=(1+gt); N*=(1+gt); N-=N*(MGMT/12); B*=(1+bt)
    if dt.month==12 or dt==df.index[-1]:
        rel=N/B
        if rel>HWM: N-=PERF*(rel-HWM)*B; HWM=N/B
    gn.append(G); nn.append(N)
gross_nav=pd.Series(gn,index=df.index); net_nav=pd.Series(nn,index=df.index)
def nm(nav):
    r=nav.pct_change().dropna(); cagr=nav.iloc[-1]**(12/len(nav))-1
    ex=r-rf.reindex(r.index).fillna(0); sh=(ex.mean()*12)/(ex.std()*np.sqrt(
    return {"CAGR":cagr,"Sharpe":sh,"MaxDD":dd}
fee=pd.DataFrame({"Gross strategy":nm(gross_nav),"NET to investor":nm(net_na
for col in ["CAGR","MaxDD"]: fee[col]=(fee[col]*100).round(1).astype(str)+"%
fee["Sharpe"]=fee["Sharpe"].round(2)
print("Terms: 1.0% management + 15% of return ABOVE the S&P 500, relative hi
fee
```

```
Terms: 1.0% management + 15% of return ABOVE the S&P 500, relative high-wate
r mark, monthly liquidity. SIMULATED net-of-fee:
```

Out[12]:

	CAGR	Sharpe	MaxDD
Gross strategy	14.9%	1.09	-21.2%
NET to investor	13.1%	0.96	-23.5%
S&P 500	10.0%	0.60	-50.8%

12 · Conclusion & honest limitations

Result (SIMULATED, 2002–2026, net of 15 bps, gross of fund fees): market-beating compounding at roughly two-thirds of the market's volatility and less than half its drawdown, with statistically significant alpha; net of all fund fees the investor still beats the S&P 500 at a higher Sharpe and lower drawdown.

The edge is capital preservation, not a faster signal — the macro overlay cuts equity exposure in stress and the defensive sleeve earns in risk-off periods.

Honest limitations (disclosed):

- Returns are net of trading costs but **gross of fund fees and taxes** (net-of-fee shown separately).
- The **alpha (+5.4%) is partly the defensive sleeve's bond/gold return**, which the FF5+Momentum model doesn't span — the cleanest wins are the Sharpe and drawdown.
- The **defensive sleeve is correlation-sensitive**: 2022 broke the bond hedge (sleeve leg negative), though the overlay alone still protected. Part of its historical benefit came from a multi-decade bond tailwind that may not repeat.
- Out-of-sample Sharpe is modestly **below** in-sample (no collapse, but disclosed).
- **All performance is SIMULATED; no live track record exists.** The fund is fictional.

Disclaimer: Fictional pitch for the ESADE Asset Management course. Not investment advice. All performance is simulated; past performance does not indicate future results.